Emerging Trends and Challenges Toward LLM Agents in Static Analysis

Hoang-Quoc-Bao Hua[1,2] and Xuan-Bach Le[1,2*]

[1] Faculty of Computer Science and Engineering, Technology (HCMUT), Ho Chi Minh City, Vietnam
[2] Vietnam National University Ho Chi Minh City, Ho Chi Minh City, Vietnam
{hhqbao.sdh241,lexuanbach}@hcmut.edu.vn

Abstract. Static analysis remains crucial for ensuring software reliability and security, yet it frequently struggles with false positives, rigid rule sets, and an inability to reason about semantic context. LLM-based agents show promise as a complement to traditional approaches by bringing autonomous reasoning and verification capabilities to the analysis pipeline. This paper introduces a taxonomy that organizes these agents along four key dimensions: architecture, integration patterns, grounding mechanisms, and application domains. Our framework captures how agents are structured, where they intervene in existing workflows, how they validate their outputs, and which tasks they address–from defect detection and code maintenance to security auditing. We examine current evaluation practices and surface major limitations around reliability and scalability. The paper concludes by distilling lessons from recent work, highlighting open research challenges, and identifying promising directions for future investigation.

Keywords: Static Program Analysis · LLM Agents · Neuro-Symbolic AI · Automated Program Repair · Software Security

1 Introduction

Software development is often seen as writing new code, but studies show developers spend most of their time understanding and reasoning about existing code [45,24]. In practice, they act as human static analyzers, tracing data flows, simulating behavior, and inferring intent in incomplete or evolving systems [25].

Despite tool advances, static analysis (SA) still lags behind this kind of contextual, adaptive reasoning. Static Application Security Testing (SAST) tools like SonarQube [48], CodeQL [13], and Infer [34] are widely used for compliance and large-scale vulnerability detection. SonarQube relies on rule-based checks, CodeQL queries program databases for known patterns, and Infer applies abstract interpretation to find runtime risks. To maintain soundness, these tools often over-approximate behavior, resulting in high false-positive rates and alert fatigue [16,26]. They also struggle with incomplete code, implicit assumptions, and complex logic where contextual, hypothesis-driven reasoning is needed [35,20].

* Corresponding author

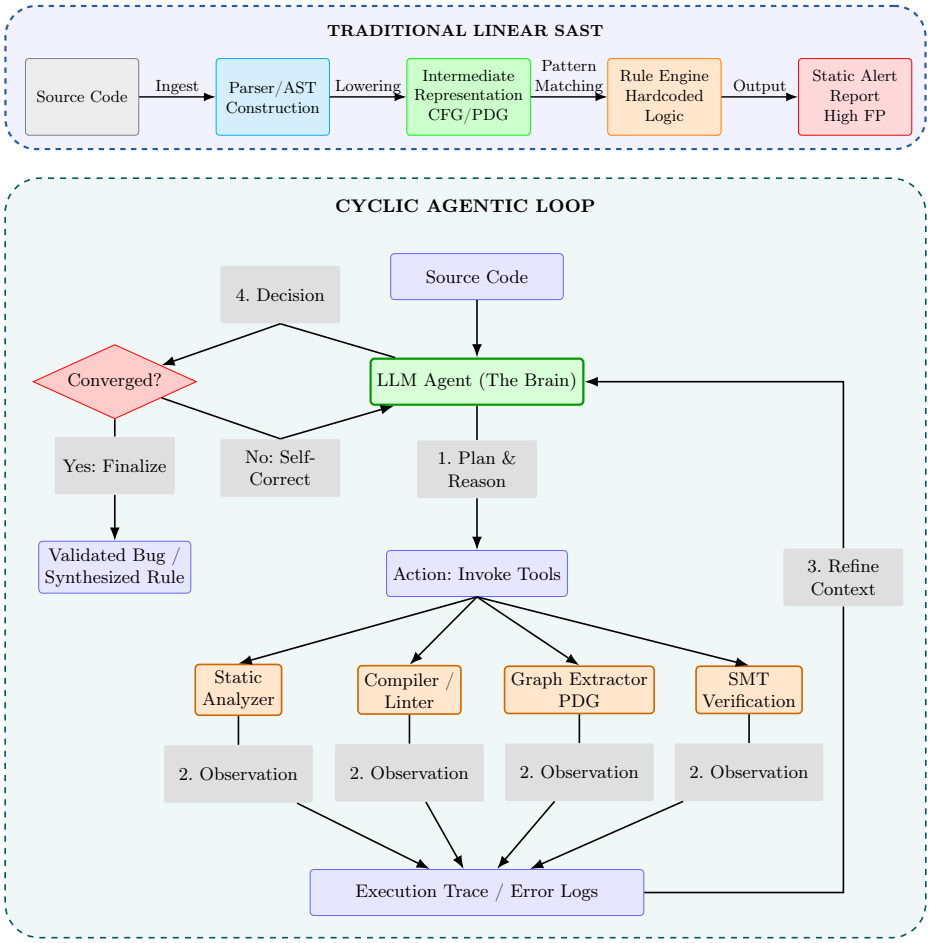

Fig. 1: The shift from linear, rule-based scanning to iterative, agentic reasoning loops with external feedback (compiler, graphs, SMT, logs) to refine and validate results.

Recent advances in large language models (LLMs) have led to the emergence of *agentic* SA workflows that aim to bridge this gap. Rather than performing single-pass classification or pattern matching, LLM-based agents plan, invoke external tools, interpret feedback, and iteratively refine their conclusions. Figure 1 illustrates this shift from linear, rule-based scanning to multi-step reasoning loops. Frameworks such as ReAct [59], ToolLLM [40] and AgentTuning [61] demonstrate how feedback and control loops enable models to navigate large search spaces, coordinate symbolic tools, and improve decision quality beyond what is possible in a single analysis pass.

In practice, LLM-based agents are increasingly employed in SA pipelines to triage warnings, localize defects, and suggest repairs. Yet, even when paired with formal verification backends, these systems often exhibit subtle failure modes that elude standard correctness metrics. Patches may satisfy verification condi-

tions while implicitly restricting input domains; inferred invariants can discharge proofs but encode unintended assumptions about the environment; and agentic workflows may converge on formally valid, yet operationally unsafe, solutions.

Our key observation is that failures in LLM-assisted SA often stem not from faulty reasoning, but from implicit assumptions introduced by LLMs and left unchecked by existing workflows. These assumptions (inputs, environments, libraries) are seldom made explicit or validated, even with formal verifiers. Hence, artifacts deemed "verified" may still behave incorrectly in deployment.

This paper examines LLM-based agents for SA through this assumption-centric lens. Rather than surveying all LLM use in software engineering, we focus on agentic systems that combine planning, tool use, and multi-step reasoning with SA backends. We introduce a focused workflow taxonomy that clarifies how agents are structured, where they intervene, and how their outputs are validated. Through this lens, we examine representative systems, surface common assumption-related failures, and distill design lessons for building deployable, verifier-in-the-loop agents.

Concretely, we make the following contributions:

- We propose a taxonomy of LLM-based SA agents, emphasizing how neural generation and symbolic verification divide responsibilities (Section 3).
- We present an assumption-aware analysis of how current pipelines introduce, propagate, and fail to manage implicit assumptions (Section 4).
- We distill design principles and outline open challenges for building reliable, certifiable, human-in-the-loop LLM-assisted analysis systems (Section 5).

We use SA to denote compile-time program analysis techniques in general, and SAST to refer to security-focused commercial tools. We define *LLM-based agents* as systems with explicit control loops and external tool integration, distinguishing them from single-pass LLM classifiers. While prior surveys cover LLMs in software engineering [11], learning-based code analysis [4], and autonomous agents [51], they do not focus on assumption management in agentic SA pipelines. By treating assumptions as a first-class design concern, this paper aims to bridge the gap between LLM capabilities and practical verification needs, offering concrete guidance for applied intelligent systems.

The rest of the paper is organized as follows. Section 2 reviews SA and the rise of LLM-based agents. Section 3 introduces our workflow taxonomy, detailing how agents are composed, integrated with tools, and grounded through verification. Section 4 applies this taxonomy to representative systems, highlighting recurring assumption-related failure modes. Section 5 discusses key evaluation gaps and deployment challenges. Finally, Section 6 distills the key lessons from our analysis and highlights open challenges and emerging research directions.

2 Background

This section reviews the traditional static analysis pipeline and examines how agent-based extensions are changing it. While SA offers scalable, compile-time

checking, its reliance on over-approximation creates difficulties with understanding developer intent, managing false positives, and adapting to code that evolves rapidly. LLM-based agents bring a more flexible, context-aware alternative that can better handle real-world complexity and uncertainty.

2.1 Limits of Traditional Static Analysis

Production static analysis tools—Infer, CodeQL, SonarQube—operate through fixed pipelines built on manually crafted rules and conservative reasoning strategies designed to preserve soundness [5,33]. Because missing a potential issue is considered unacceptable, these tools rely on *over-approximation* [6]: when a code path is uncertain or ambiguous, they treat it as though it might cause an error. This approach enables scalability and broad coverage, but it comes at a cost. Benign code often gets flagged as risky, contributing to high false-positive rates and developer burnout from constant, low-value alerts [5].

This soundness-precision trade-off has been documented extensively. Johnson et al. [20] found that false-positive rates in practice frequently exceed 70%, while Du et al. [9] revealed the substantial manual triage burden such warnings impose in industrial contexts. Groce et al. [15] went further, showing that chronic false positives damage developer trust and encourage widespread warning suppression. Precision is not the only issue. Murphy et al. [35] demonstrated that static analyzers struggle to capture implicit semantic intent and contextual assumptions, and Samhi et al. [43] empirically found that many analyzers miss a large fraction of code that actually executes at runtime. Li et al. [30] concluded that these fundamental precision, semantic, and coverage limitations persist even in large-scale, cutting-edge analyzers. Taken together, these findings suggest that traditional static analysis faces inherent constraints that cannot be overcome simply by scaling up.

2.2 LLM-Based Agents for Static Analysis

Recent research explores whether LLM-based agents can address some of these limitations. Agents differ from single-pass classifiers in a fundamental way: they integrate language models with control loops that enable iterative reasoning, planning, and tool use [46]. An agent can generate hypotheses, invoke analyzers or solvers to test them, and refine its conclusions based on the feedback it receives. This pattern appears in systems that train models to plan and execute API calls [44] and in surveys of tool-augmented agent architectures [46].

One widely adopted approach builds on the ReAct-style reasoning loop [59], which interleaves reasoning steps with tool invocations to test and refine hypotheses iteratively. MRKL systems [22] route different subtasks to specialized tools, while Toolformer [44] teaches models when and how to make those calls. Gorilla [38] improves the accuracy of structured function invocations. These architectural choices let agents validate or refine static analysis results, often coordinating multiple tools—from analyzers to SMT solvers or debuggers—to achieve deeper verification.

When tasks grow more complex, multi-agent systems can divide labor across specialized roles such as generation, validation, and evaluation, boosting both robustness and coverage [18,51]. Recent work has extended reactive loops with planning and reflection capabilities, improving performance on multi-step decision-making tasks [47]. Collectively, these advances reflect a shift away from rigid, rule-based pipelines toward more interactive, agentic workflows.

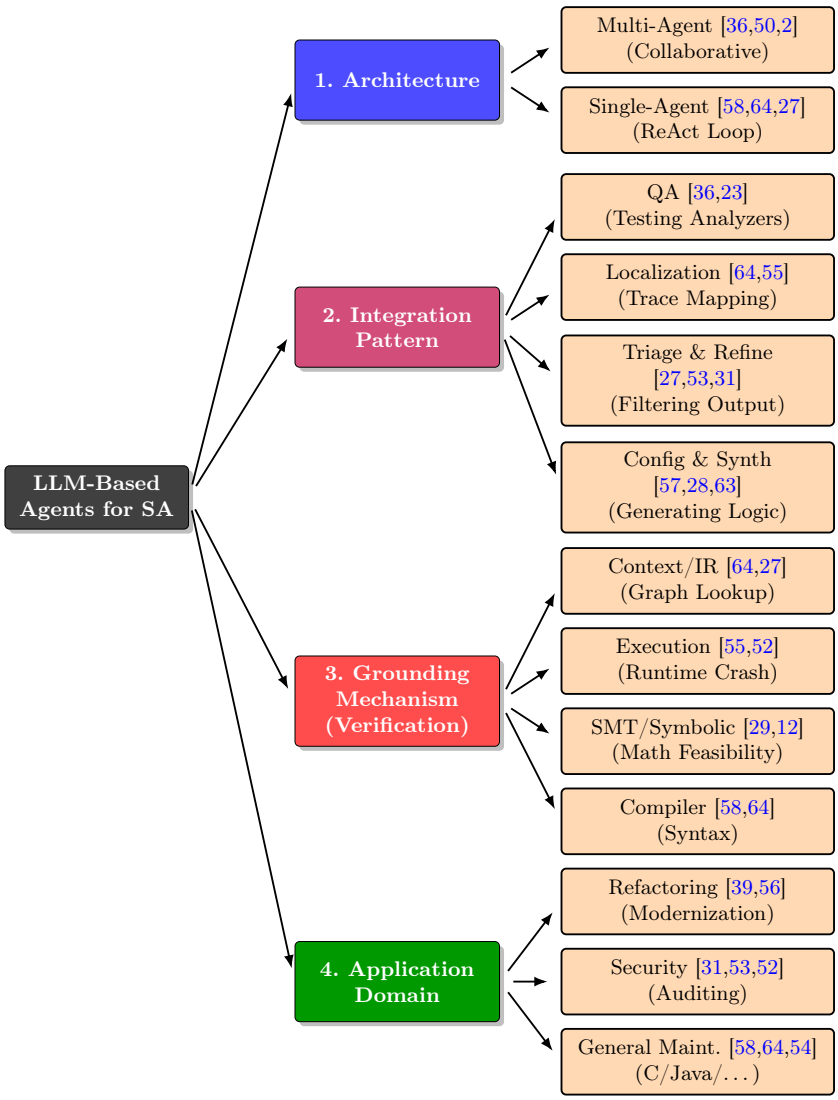

Fig. 2: Taxonomy of LLM-based agents for static analysis, organized by architecture, integration pattern, grounding mechanism, and application domain.

3 Taxonomy and Architectures

We propose a four-part framework–covering architecture, integration, grounding, and domain–to organize the emerging field of agentic static analysis (see Figure 2). Unlike prior surveys focused on capabilities or tasks, our taxonomy shows how agents are built, where they operate, and how their reasoning is verified.

3.1 Architecture

Our review suggests that single-agent architectures remain predominant which captures both reasoning granularity and the level of agent collaboration. Typically, a single LLM is paired with external tools and operates via iterative reasoning-action loops, often following the ReAct paradigm [59]. For example, ZeroFalse [53] refines SA alerts by querying context and updating judgments over multiple steps, T2L-Agent [55] uses call graphs to localize faults from runtime traces using tool-assisted reasoning.

In contrast, multi-agent architectures divide tasks across specialized roles to handle complexity beyond a single model's capacity. StaAgent [36] uses a multi-stage pipeline where separate agents generate, mutate, and validate program variants to test analyzers via metamorphic relations. MAGIS [50] coordinates multiple agents to reason over large codebases and resolve cross-file issues in collaborative development workflows. These architectures demonstrate how agent design choices, from single-agent loops to multi-agent pipelines, shape the flexibility, scalability, and depth of reasoning in SA systems.

3.2 Integration Pattern

This dimension reflects how the agent works alongside existing static analysis tools. Some agents act upstream, generating the logic those tools rely on, like rules or checkers. For instance, KNighter [57] learns to create checkers from past bug-fix commits, while RuleLLM [63] generates YARA and Semgrep rules to detect malware or code patterns.

Triage and refinement agents work after static analyzers, helping to filter or validate their outputs. LLift [27] uses contextual reasoning to suppress infeasible warnings, IRIS [31] evaluates whether findings align with specs, and ZeroFalse [53] applies step-by-step reasoning to identify false positives. Localization agents link runtime signals to static code–T2L-Agent [55] follows execution traces to find faults, while AutoCodeRover [64] uses test feedback to track down root causes. To test analyzer reliability, StaAgent [36] generates adversarial code variants that expose inconsistencies. Together, these agents show how LLMs can move static analysis from rigid rule-checking to a more flexible, context-aware process that better supports real-world software development.

Table 1: Representative LLM-based agents for SA using the proposed taxonomy.

System	Architecture	Integration	Grounding	Domain	Key insight
KNighter [57]	Single-agent	Logic synthesis	Compiler	Tool dev.	Learns checkers from patches
RuleLLM [63]	Single-agent	Rule synthesis	Compiler	Security	Iterative rule repair
LLift [27]	Single-agent	Triage	Constraints	Security	Feasibility-based FP reduction
ZeroFalse [53]	Single-agent	Triage	Context + trace	Security	Contract-style warning verification
IRIS [31]	Single-agent	Triage	Spec inference	Security	Library-aware taint reasoning
T2L-Agent [55]	Hierarchical	Localization	Runtime trace	Security	Maps crashes to static code
AutoCodeRover [64]	Single-agent	Localization	Execution	Maintenance	Test-guided navigation
ConSynergy [12]	Hybrid	Verification	SMT	Security	SMT-based path reasoning
AutoBug [29]	Hybrid	Symbolic exec.	SMT	Maintenance	LLM-assisted symbolic execution
StaAgent [36]	Multi-agent	Tool testing	Metamorphic	Tool dev.	Finds analyzer inconsistencies

3.3 Grounding Mechanism

Grounding mechanisms help agents validate their reasoning and avoid hallucinations. Compiler-based grounding uses feedback from compilation to ensure syntactic and type correctness, especially in rule and checker generation. Symbolic grounding translates program behavior into logical constraints for SMT solvers–ConSynergy [12] checks inferred interactions this way, while AutoBug [29] confirms bug feasibility via symbolic execution.

Execution-based grounding verifies agent reasoning by running tests, code, or debuggers. SWE-agent [58] iteratively edits code and runs tests to check correctness. In contrast, graph-based grounding relies on static structures like call or dependence graphs to provide contextual reasoning without execution, as seen in AgentArmor [52] for security analysis. These grounding strategies help improve agent reliability, supporting more trustworthy and context-aware SA.

3.4 Application Domain

We group systems according to their primary application domain. General maintenance systems target tasks such as bug fixing, fault localization, and refactoring across large codebases; security auditing systems focus on vulnerability detection, triage, and false-positive reduction; and tool development systems aim to construct, test, or improve static analyzers themselves. Table 1 summarizes representative LLM-based agents within each domain and situates them along the proposed architectural, integration, and grounding dimensions.

Agents in this taxonomy span a wide range of goals and techniques. KNighter [57] and RuleLLM [63] concentrate on tool generation, creating checkers or refining detection rules based on past vulnerabilities. LLift [27], ZeroFalse [53], and IRIS [31] aim to improve security auditing by filtering out false positives through contextual reasoning or constraint analysis. T2L-Agent [55] and AutoCodeRover [64] support fault localization by bridging dynamic signals and static code. ConSynergy [12] and AutoBug [29] validate program paths using symbolic execution and

constraint solving. StaAgent [36] enhances tool robustness by generating metamorphic test cases that reveal inconsistencies. Together, these systems demonstrate how agentic approaches can extend static analysis with more adaptive, context-aware, and verifiable reasoning throughout the software lifecycle.

4 Applications and Implementation Strategies

Building on our taxonomy, this section examines how LLM-based agents get applied across the static analysis lifecycle. We organize the literature around the primary capabilities that agentic reasoning enables: synthesizing rules, autonomously localizing defects, adjudicating false positives, and performing meta-analysis on analysis tools themselves.

4.1 Rule Synthesis and Configuration

One significant obstacle to adopting advanced static analysis tools lies in mastering their domain-specific languages and configurations [20]. Agentic systems tackle this "input problem" by converting intent into formal analysis logic.

Patch-based synthesis takes one approach: it infers safety invariants from historical fixes. KNighter [57], for instance, studies pairs of vulnerable and patched code to synthesize static analysis checkers for the Clang Static Analyzer, showing that learned rules can generalize beyond their training data. A complementary line of work focuses on text-to-query generation, where agents translate natural language vulnerability descriptions into executable queries. Systems like MoCQ [28] and RuleLLM [63] combine retrieval with iterative repair to produce valid CodeQL or YARA rules, reducing the expertise barrier for writing precise analyses. These approaches demonstrate how agentic reasoning can bridge the gap between what developers intend and what formal tools require, making static analysis more accessible and adaptable.

4.2 Autonomous Localization and Navigation

Fault localization poses challenges for traditional tools that depend on complete builds or global program models. Agentic systems take a different path: they perform iterative, demand-driven exploration guided by whatever partial evidence is available.

Trace-guided approaches leverage runtime signals to focus static analysis. T2L-Agent [55], for example, follows crash traces through syntax trees to localize faults without needing to scan the entire codebase. Other systems, such as AutoCodeRover [64], separate navigation from deep reasoning. They first identify promising regions using textual and structural clues, then concentrate detailed analysis on smaller code slices. This separation helps agents manage large codebases without losing the broader context. These methods illustrate how runtime evidence can steer agents toward faster, more targeted fault localization.

4.3 False Positive Adjudication

In industrial practice, the most immediate application for LLM-based agents is reducing false positives [10]. Here, agents serve as semantic adjudicators: they reason over the results from static analysis tools rather than attempting to replace those tools outright.

One strategy involves post-constraint reasoning. LLift [27], for instance, analyzes path conditions and implicit checks that traditional analyzers find difficult to encode. Contract-based methods take reported warnings as hypotheses that need verification. ZeroFalse [53] attempts to construct feasible counterexamples or prove infeasibility using contextual and trace information, suppressing alerts when no valid execution path exists. Other systems like IRIS [31] focus on specification inference, dynamically enriching analyzer knowledge by learning missing sanitization rules for custom libraries. Together, these approaches show how agentic reasoning can augment traditional analyzers–validating, refining, or dismissing alerts through deeper semantic and contextual analysis.

4.4 Meta-Analysis of Static Analyzers

A smaller but expanding body of work applies agents to evaluate the reliability of static analyzers themselves. StaAgent [36] exemplifies this approach through agent-driven metamorphic testing. It generates semantically equivalent program variants and compares how analyzers treat them, exposing brittle rules and inconsistent behavior. This application of agents shifts static analysis away from being a fixed, unchanging pipeline toward a process that can self-evaluate and improve continuously.

5 Evaluation Challenges and Benchmarking

The shift from deterministic static analyzers to probabilistic, agentic systems raises fundamental challenges for evaluation. Traditional benchmarks and metrics, designed for fixed analysis pipelines, do not adequately capture iterative reasoning, tool interaction, or verification behavior.

5.1 From Snippet-Level to Repository-Level Evaluation

Early benchmarks like HumanEval [3] and MBPP [1] focus on isolated functions, offering limited insight into context retention or cross-file reasoning. Newer benchmarks, such as SWE-bench [8] and Defects4J [21], operate at the repository level but mainly address automated repair, not detection or verification. LoCoBench [41] introduces bias-free metrics like *Information Coverage* and *Long-Range Dependency Resolution* to assess codebase exploration, while Reliability-Bench [17] uses fault injection to evaluate robustness. Industrial efforts prioritize cost and false-positive reduction [10]. Still, there are no benchmarks that capture the full complexity of agentic SA, including tool coordination, multi-step reasoning, and assumption handling.

5.2 Metrics for Agentic Static Analysis

Evaluating agentic systems requires metrics that reflect both outcomes and the quality of reasoning behind them. Standard measures like precision and recall are often too coarse. *Pairwise Accuracy* [60] tests if agents can distinguish vulnerable from fixed code, while *False Positive Reduction* [10] assesses noise suppression without missing true issues. *Smell Reduction Rate* [10] gauges whether refactorings lead to real quality gains.

Reliability and behavior are equally important. ReliabilityBench [17] introduces a *Reliability Surface* to assess consistency, robustness, and fault tolerance across runs. Even deterministic settings show variance, captured by *Determinism Correlation*. Workflow-level metrics from LoCoBench-Agent [41], like *Information Coverage*, *Long-Range Dependency Resolution*, and *Tool Success Rate*, help evaluate how agents reason over time.

Finally, cost is a key constraint. Agentic workflows are $10 - 15$ times more expensive than traditional tools [62], so metrics like token-to-bug ratio, interaction depth, and per-alarm cost [10] matter. The LLM-Verifier Convergence Theorem [7] offers guidance for managing compute budgets.

5.3 Deployment Barriers

Agentic systems often exhibit $15 - 25\%$ classification variance across runs [37], which creates reproducibility challenges. Techniques like ensemble voting and conservative decoding can reduce this variance, though they increase computational costs. These systems also remain vulnerable to indirect prompt injection through adversarial code comments [14]. While defenses such as AgentArmor [52] help, adaptive attacks continue to evolve, making human oversight necessary. Scalability presents another limitation: large codebases frequently exceed context windows [32], forcing systems to rely on retrieval-based fragmentation.

A significant deployment barrier lies in designing effective human-in-the-loop interventions that satisfy strict regulatory standards without negating the scalability benefits of agentic systems. Safety-critical domains governed by standards like DO-178C [42] and IEC 62304 [19] require deterministic behavior that current LLM agents cannot guarantee. These agents are better positioned as assistive tools that produce verifiable outputs, given known failure modes such as hallucinated vulnerabilities and excessive reliance on surface-level code patterns.

Operationalizing this oversight is challenging. A compliant deployment workflow must orchestrate a complex, three-stage handshake: first, during agentic triage, the LLM must be constrained to isolate and explicitly output its operational assumptions [59]. Next, the system must seamlessly route these assumptions through an automated grounding phase to attempt mathematical verification via SMT solvers or compilers [29]. Finally, human intervention must be strictly confined to auditing only the unverified assumptions, feeding any corrections back into the agent's context for a new reasoning loop [49]. Developing platforms capable of reliably managing this structured assumption-adjudication loop—without introducing new context-management failures or overwhelming the human operator—remains a major operational hurdle.

6 Lessons Learned and Open Challenges

Lessons Learned. Recent work shows that LLM-based agents can genuinely enhance static analysis by adding semantic reasoning, contextual interpretation, and iterative refinement. Empirical studies document improvements in warning triage, false positive reduction, and analyst productivity. Yet these gains emerge primarily when agents *augment* established tools rather than replacing them outright. In practice, systems depend on analyzers like CodeQL or Infer to produce initial findings, with LLMs functioning as adaptive layers that interpret and prioritize results. This reveals an important insight: sound static analyzers remain required for ensuring correctness, while LLM agents currently serve as complements that enhance precision and usability.

Open Research Challenges. Moving from prototypes to production-ready systems requires addressing several key challenges. Scalability stands out as a significant limitation: most agentic architectures have been tested on relatively small tasks, and how they scale to large codebases–without hitting context limits or coordination bottlenecks–remains unclear. Reliability and grounding are equally critical. Agents reason heuristically, so their outputs must align closely with formal tools to avoid producing brittle or misleading results. Deeper integration with SMT solvers, abstract interpreters, and proof engines–where agents guide search or propose invariants–has yet to be fully explored.

Evaluation presents another gap. Current benchmarks emphasize detection accuracy but neglect workflow performance, robustness, and cost-effectiveness. We need realistic, end-to-end benchmarks that capture how agents behave under practical constraints. Deployment adds its own regulatory and operational hurdles: agents must support auditability, provide fallback mechanisms, and comply with standards–particularly in safety-critical contexts.

Emerging Trends and Outlook. The field is shifting away from single-pass LLM use toward more interactive, verifier-in-the-loop agentic workflows that emphasize iteration and tool coordination. Agents themselves are evolving–from simple patch generators into analysis coordinators that bridge symbolic tools and human oversight. We also see growing attention to evaluation rigor and deployment constraints, reflecting a maturation toward real-world applicability.

LLM-based agents offer a promising route toward static analysis that is more adaptive and context-aware. Realizing their full potential will require assumption-aware design, tighter integration with formal verification tools, and evaluation frameworks grounded in realistic scenarios. These steps can lay the foundation for robust, next-generation analysis systems.

Recommended Safeguards for Agents Design Looking across the systems we analyzed, several essential safeguards stand out. To make future systems more reliable and reduce the risks of hidden assumptions, we recommend three core design principles. First, agents must explicitly state their environmental and logical assumptions before generating any code, turning those hidden thoughts into checkable artifacts. Second, systems should use multi-modal grounding—meaning they

cross-check static constraints (like SMT solvers) against dynamic validation (like execution traces) to expose any operational blind spots. Finally, agents need built-in fail-safes; if their confidence drops, they should gracefully fall back to a cautious, traditional analysis and escalate the issue to a human reviewer.

Disclaimer. The authors used AI-based tools as an assisted editing tool, to improve spelling, grammar, clarity, and readability during manuscript preparation. After using this tool, the authors carefully reviewed, edited the text and take full responsibility for the final content.

Acknowledgments

We acknowledge Ho Chi Minh City University of Technology (HCMUT), VNUHCM for supporting this study.

References

1. Nye, M., Odena, A., Austin, J., et al.: Program synthesis with large language models. arXiv preprint arXiv:2108.07732 (2021)
2. Batole, F., Ullah, M.R., Bellur, A., et al.: Multi-agent coordinated rename refactoring. arXiv preprint arXiv:2601.00482 (2026)
3. Chen, M.: Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374 (2021)
4. Chen, Z., Monperrus, M.: A survey of deep learning-based source code analysis. ACM Comput. Surv. (2022)
5. Chess, B., West, J.: Secure Programming with Static Analysis. Addison-Wesley Professional (2007)
6. Cousot, P., Cousot, R.: Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In: Proc. POPL. pp. 238–252 (1977)
7. Sun, Y., Cordeiro, L., Dantas, P., et al.: The $4/\delta$ bound: Designing predictable LLM-verifier systems for formal method guarantee
8. Pan, E., Da, J., Deng, X., et al.: SWE-Bench pro: Can ai agents solve long-horizon software engineering tasks? arXiv preprint arXiv:2509.16941 (2025)
9. Sui, Y., Liu, Y., Du, S., et al.: LEOPARD: Identifying vulnerable code for vulnerability assessment through program metrics. IEEE Trans. Softw. Eng. (2019)
10. Zou, Y., Feng, J., Du, X., et al.: Reducing false positives in static bug detection with LLMs: An empirical study in industry. arXiv preprint arXiv:2501.18844 (2025)
11. Harman, M., Gokkaya, B., Fan, A., et al.: Large language models for software engineering: Survey and open problems. Proc. ICSE-FoSE (2023)
12. Zhang, K., Chen, Y., Feng, Z., et al.: Concurrency bug detection via static analysis and large language models. Future Internet (2025)
13. GitHub: CodeQL. https://codeql.github.com/ (2023)
14. Mishra, S., Abdelnabi, S., Greshake, K., et al.: Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. Proc. ACM Workshop AISec (2023)
15. Holzmann, G., Havelund, T., Groce, A., et al.: What went wrong: Explaining counterexamples. In: Proc. SPIN (2012)

16. Liu, S., Tan, T., Guo, Z., et al.: Mitigating false positive static analysis warnings: Progress, challenges, and opportunities. IEEE Trans. Softw. Eng. (2023)
17. Gupta, A.: ReliabilityBench: Evaluating LLM agent reliability under production-like stress conditions. arXiv preprint arXiv:2601.06112 (2025)
18. Chen, J., Zheng, X., Hong, S., et al.: MetaGPT: Meta programming for a multi-agent collaborative framework. arXiv preprint arXiv:2308.00352 (2023)
19. International Electrotechnical Commission: Iec 62304:2006/amd 1:2015, medical device software — software life cycle processes. Tech. rep. (2015)
20. Murphy-Hill, E., Song, Y., Johnson, B., et al.: Why don't software developers use static analysis tools to find bugs? Proc. ICSE pp. 672–681 (2013)
21. Just, R., Jalali, D., Ernst, M.D.: Defects4J: A database of existing faults to enable controlled testing studies for java programs. In: Proc. ISSTA. pp. 437–440 (2014)
22. Aharonov, R., Levy, O., Karpas, E., et al.: MRKL systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources, and discrete reasoning. arXiv preprint arXiv:2205.00445 (2022)
23. Sani, M.F., Hu, R., Keltek, M., et al.: LSAST: Enhancing cybersecurity through LLM-supported static application security testing. In: IFIP Int. Conf. ICT Syst. Secur. Privacy Prot. pp. 166–179 (2025)
24. Ko, A.J., Myers, B.A., Aung, H.H.: Six learning barriers in end-user programming systems. In: Proc. IEEE VLHCC. pp. 199–206 (2004)
25. LaToza, T.D., Venolia, G., DeLine, R.: Maintaining mental models: A study of developer work habits. In: Proc. ICSE. pp. 492–501 (2006)
26. Zhai, Y., Hao, Y., Li, H., et al.: Assisting static analysis with large language models: A ChatGPT experiment. In: Proc. ESEC/FSE. pp. 2107–2111 (2023)
27. Zhai, Y., Hao, Y., Li, H., et al.: Enhancing static analysis for practical bug detection: An LLM-integrated approach. Proc. PACMPL (2024)
28. Korich, J.S., Yao, S., Li, P., et al.: Automated static vulnerability detection via a holistic neuro-symbolic approach. arXiv preprint arXiv:2504.16057 (2025)
29. Li, Y., Meng, R., Duck, G.J.: Large language model powered symbolic execution. Proc. PACMPL (2025)
30. Li, Y., Lo, D., Shihab, E.: Large-scale empirical study of static analysis tools. Empir. Softw. Eng. (2021)
31. Li, Z., Dutta, S., Naik, M.: IRIS: LLM-assisted static analysis for detecting security vulnerabilities. arXiv preprint arXiv:2405.17238 (2025)
32. Hewitt, J., Lin, K., Liu, N.F., et al.: Lost in the middle: How language models use long contexts. Trans. Assoc. Comput. Linguist. (2024)
33. Livshits, V.B., Lam, M.S.: Finding security vulnerabilities in java applications with static analysis. In: Proc. USENIX Security. pp. 271–286 (2005)
34. Meta (formerly Facebook): Infer. https://fbinfer.com/ (2023)
35. Murphy-Hill, E., Parnin, C., Black, A.P.: How we refactor, and how we know it. IEEE Trans. Softw. Eng. (2018)
36. Shin, J., Ahmed, M.B.U., Nnorom, E., et al.: StaAgent: An agentic framework for testing static analyzers. arXiv preprint arXiv:2507.15892 (2025)
37. Harman, M., Zhang, J.M., Ouyang, S., et al.: LLM is like a box of chocolates: the non-determinism of ChatGPT in code generation. arXiv preprint arXiv:2308.02828 (2023)
38. Ma, X., Zhang, T., Patil, S., et al.: Gorilla: Large language model connected with massive apis. arXiv preprint arXiv:2305.15334 (2023)
39. Dilhara, M., Bellur, A., Pomian, D., et al.: EM-assist: Safe automated extract-method refactoring with LLMs. In: Proc. FSE Companion (2024)

40. Ye, Y., Liang, S., Qin, Y., et al.: ToolLLM: Facilitating large language models to master 16000+ real-world APIs. arXiv preprint arXiv:2307.16789 (2023)
41. Liu, Z., Liu, Z., Qiu, J., et al.: LoCobench: A benchmark for long-context large language models in complex software engineering. arXiv:2509.09614 (2025)
42. RTCA: Do-178c, software considerations in airborne systems and equipment certification. Tech. rep., Radio Technical Commission for Aeronautics (RTCA) (2011)
43. Bissyandé, T.F., Just, R., Samhi, J., et al.: Call graph soundness in android static analysis. arXiv preprint arXiv:2407.07804 (2024)
44. Dessì, R., Dwivedi-Yu, J., Schick, T., et al.: Toolformer: Language models can teach themselves to use tools. arXiv preprint arXiv:2302.04761 (2023)
45. Seaman, C.B.: Qualitative methods in empirical studies of software engineering. IEEE Trans. Softw. Eng. (1999)
46. Shen, Z.: LLM with tools: A survey. arXiv preprint arXiv:2409.18807 (2024)
47. Gopinath, A., Cassano, F., Shinn, N., et al.: Reflexion: Language agents with verbal reinforcement learning (2023)
48. SonarSource: SonarQube. https://www.sonarqube.org/ (2023)
49. Thongtanunam, P., Pasuksmit, J., Takerngsaksiri, W., et al.: Human-in-the-loop software development agents. pp. 342–352. IEEE (2025)
50. Wang, Y., Zhou, Y., Tao, W., et al.: MAGIS: Llm-based multi-agent framework for github issue resolution. In: Proc. NeurIPS (2024)
51. Feng, X., Ma, C., Wang, L., et al.: A survey on large language model based autonomous agents. Front. Comput. Sci. (2024)
52. Lu, Y., Liu, Y., Wang, P., et al.: Securing large language model agents via structured graph abstraction. arXiv preprint arXiv:2508.01249 (2025)
53. Wang, S., et al.: ZeroFalse: Improving precision in static analysis with LLMs. arXiv preprint arXiv:2510.02534 (2024)
54. Wei, Y., Xia, C.S., Zhang, L.: Copiloting the copilots: Fusing large language models with completion engines for automated program repair. Proc. ESEC/FSE (2023)
55. Dolan-Gavitt, B., Shao, M., Xi, H., et al.: From trace to line: LLM agent for real-world OSS vulnerability localization. arXiv preprint arXiv:2510.02389 (2025)
56. Yang, J., Lin, F., Xu, Y., et al.: MANTRA: Enhancing automated method-level refactoring with contextual rag and multi-agent LLM collaboration. arXiv preprint arXiv:2503.14340 (2025)
57. Xie, Z., Zhao, Z., Yang, C., et al.: Knighter: Transforming static analysis with llm-synthesized checkers. In: Proc. SOSP (2025)
58. Wettig, A., Jimenez, C.E., Yang, J., et al.: SWE-agent: Agent-computer interfaces enable automated software engineering. Proc. NeurIPS (2024)
59. Yu, D., Zhao, J., Yao, S., et al.: React: Synergizing reasoning and acting in language models. In: The eleventh international conference on learning representations (2022)
60. Lou, Y., Teo, S.G., Yildiz, A., et al.: Benchmarking LLMs and LLM-based agents in practical vulnerability detection for code repositories. In: Proc. ACL (2025)
61. Lu, R., Liu, M., Zeng, A., et al.: Agenttuning: Enabling generalized agent abilities for llms. In: Proc. ACL (2024)
62. Zhang, Q., Wornow, M., Olukotun, K.: Agentic plan caching: Test-time memory for fast and cost-efficient LLM agents. In: Proc. NeurIPS (2025)
63. Chen, H., Du, X., Zhang, X., et al.: Automatically generating rules of malicious software packages via large language model. In: Proc. DSN (2025)
64. Fan, Z., Ruan, H., Zhang, Y., et al.: AutoCodeRover: Autonomous program improvement. Proc. ISSTA (2024)